

Министерство науки и высшего образования РФ
ФГАОУ ВПО
Национальный исследовательский технологический университет
«МИСИС»

Институт информационных технологий и компьютерных наук (ИТКН)
Кафедра инфокоммуникационных технологий (ИКТ)

Отчет по лабораторной работе № 10
по дисциплине «Объектно-ориентированное программирование»
на тему «Многопоточное программирование в C#»

Выполнил:
студент группы БИВТ-25-1
Кирюшин А. А.
Проверил:
доцент каф. ИКТ
Стучилин В.В.

Москва, 2026

1. ЦЕЛЬ И ЗАДАЧИ РАБОТЫ

Цель работы:

Изучение механизмов многопоточности библиотеки TPL и низкоуровневых средств синхронизации, а также анализ масштабируемости алгоритмов при изменении вычислительных ресурсов.

Задачи:

1. Реализовать программное решение задачи о рюкзаке методом полного перебора, используя конструкцию `Parallel.For`.
2. Применить механизмы потокобезопасности (`Thread-Local Storage`) для обеспечения корректности вычислений.
3. Выполнить дополнительное задание: реализовать альтернативное решение с использованием низкоуровневого класса `Thread` с самостоятельной декомпозицией данных и синхронизацией через `lock`.
4. Провести серию замеров времени выполнения задачи для 1, 2, 4, 8, 12 и 16 потоков.
5. Построить график зависимости времени выполнения от лимита потоков, включая линию «идеального ускорения».
6. Выявить предел масштабируемости системы.

Вариант 10:

Решить задачу о рюкзаке методом полного перебора. Максимально допустимый суммарный вес рюкзака — 50 кг, выбор делается среди 25 предметов со случайными значениями веса от 1 до 5 кг и случайной ценностью от 1 до 100. Количество возможных комбинаций составляет $2^{25} = 33\,554\,432$.

2. ОСНОВНЫЕ КОМПОНЕНТЫ И ИСХОДНЫЙ КОД

2.1 Алгоритмы решения (`KnapsackSolver.cs`)

Класс инкапсулирует логику решения задачи тремя способами: последовательно, через `Parallel.For` (с использованием TLS) и через массив сырых потоков `Thread`.

Полный код: `KnapsackSolver.cs`

```
using System;
using System.Threading;
using System.Threading.Tasks;
using KnapsackApp.Models;

namespace KnapsackApp.Services;
```

```

public class KnapsackSolver
{
    private readonly Item[] _items;
    private readonly int _maxWeight;
    private readonly int _totalCombinations;

    public KnapsackSolver(Item[] items, int maxWeight)
    {
        _items = items;
        _maxWeight = maxWeight;
        _totalCombinations = 1 << items.Length;
    }

    public KnapsackResult SolveSequential()
    {
        int maxValue = 0, bestCombination = 0, bestWeight = 0;

        for (int i = 0; i < _totalCombinations; i++)
        {
            int currentWeight = 0, currentValue = 0;
            for (int j = 0; j < _items.Length; j++)
                if ((i & (1 << j)) != 0)
                    { currentWeight += _items[j].Weight; currentValue +=
_items[j].Value; }

            if (currentWeight <= _maxWeight && currentValue > maxValue)
                { maxValue = currentValue; bestCombination = i; bestWeight =
currentWeight; }
        }
        return new KnapsackResult { MaxValue = maxValue, BestCombination =
bestCombination, TotalWeight = bestWeight };
    }

    public KnapsackResult SolveParallelFor(int maxDegreeOfParallelism)
    {
        int globalMaxValue = 0, globalBestCombination = 0, globalBestWeight = 0;
        var options = new ParallelOptions { MaxDegreeOfParallelism =
maxDegreeOfParallelism };

        Parallel.For(0, _totalCombinations, options,
            () => new KnapsackResult(),
            (i, state, localResult) =>
            {
                int currentWeight = 0, currentValue = 0;
                for (int j = 0; j < _items.Length; j++)
                    if ((i & (1 << j)) != 0)
                        { currentWeight += _items[j].Weight; currentValue +=
_items[j].Value; }

                if (currentWeight <= _maxWeight && currentValue >
localResult.MaxValue)
                    { localResult.MaxValue = currentValue; localResult.BestCombination
= i; localResult.TotalWeight = currentWeight; }
                return localResult;
            },
            localResult =>

```

```

        {
            lock (this)
            {
                if (localResult.MaxValue > globalMaxValue)
                { globalMaxValue = localResult.MaxValue; globalBestCombination
= localResult.BestCombination; globalBestWeight = localResult.TotalWeight; }
            }
        });

        return new KnapsackResult { MaxValue = globalMaxValue, BestCombination =
globalBestCombination, TotalWeight = globalBestWeight };
    }

    public KnapsackResult SolveThreads(int numThreads)
    {
        int globalMaxValue = 0, globalBestCombination = 0, globalBestWeight = 0;
        object locker = new object();
        Thread[] threads = new Thread[numThreads];
        int chunkSize = _totalCombinations / numThreads;
        int remainder = _totalCombinations % numThreads;
        int startIdx = 0;

        for (int t = 0; t < numThreads; t++)
        {
            int start = startIdx;
            int end = start + chunkSize + (t < remainder ? 1 : 0);
            startIdx = end;

            threads[t] = new Thread(() =>
            {
                int localMaxValue = 0, localBestCombination = 0, localBestWeight =
0;

                for (int i = start; i < end; i++)
                {
                    int currentWeight = 0, currentValue = 0;
                    for (int j = 0; j < _items.Length; j++)
                        if ((i & (1 << j)) != 0)
                            { currentWeight += _items[j].Weight; currentValue +=
_items[j].Value; }

                    if (currentWeight <= _maxWeight && currentValue >
localMaxValue)
                        { localMaxValue = currentValue; localBestCombination = i;
localBestWeight = currentWeight; }
                }
                lock (locker)
                {
                    if (localMaxValue > globalMaxValue)
                    { globalMaxValue = localMaxValue; globalBestCombination =
localBestCombination; globalBestWeight = localBestWeight; }
                }
            });
            threads[t].Start();
        }
        foreach (var thread in threads) thread.Join();
        return new KnapsackResult { MaxValue = globalMaxValue, BestCombination =
globalBestCombination, TotalWeight = globalBestWeight };
    }
}

```

2.2 Сервис бенчмаркинга (BenchmarkService.cs)

Отвечает за прогрев (JIT warmup) и сбор метрик времени для заданного количества потоков.

Полный код: BenchmarkService.cs

```
using System;
using System.Collections.Generic;
using System.Diagnostics;
using System.Threading.Tasks;
using KnapsackApp.Models;

namespace KnapsackApp.Services;

public class BenchmarkResult
{
    public int Threads { get; set; }
    public long ParallelForTimeMs { get; set; }
    public long ThreadsTimeMs { get; set; }
}

public class BenchmarkService
{
    public static async Task<List<BenchmarkResult>> RunBenchmarksAsync(
        Item[] items, int maxWeight, int[] threadCounts, Action<string> logAction)
    {
        return await Task.Run(() =>
        {
            var solver = new KnapsackSolver(items, maxWeight);
            var results = new List<BenchmarkResult>();

            logAction("Выполняется прогрев (JIT warmup)...");
            solver.SolveParallelFor(1);
            solver.SolveThreads(1);
            logAction("Прогрев завершен.\n");

            logAction("Запуск последовательного алгоритма...");
            var sw = Stopwatch.StartNew();
            var seqResult = solver.SolveSequential();
            sw.Stop();
            logAction($"Последовательный: {sw.ElapsedMilliseconds} мс. Макс.
ценность: {seqResult.MaxValue}\n");

            foreach (int threads in threadCounts)
            {
                logAction($"Тестирование для {threads} потоков...");

                sw.Restart();
                solver.SolveParallelFor(threads);
                sw.Stop();
                long pTime = sw.ElapsedMilliseconds;
                logAction($" Parallel.For: {pTime} мс");
            }
        });
    }
}
```

```

        sw.Restart();
        solver.SolveThreads(threads);
        sw.Stop();
        long tTime = sw.ElapsedMilliseconds;
        logAction($"  Threads: {tTime} mc\n");

        results.Add(new BenchmarkResult { Threads = threads,
ParallelForTimeMs = pTime, ThreadsTimeMs = tTime });
    }
    return results;
});
}
}

```

2.3 Интерфейс и графики (MainWindow.axaml и MainWindow.axaml.cs)

Интерфейс построен на Avalonia UI, графики отрисовываются с помощью библиотеки ScottPlot.

Полный код: MainWindow.axaml

```

<Window xmlns="https://github.com/avaloniaui"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
xmlns:vm="using:KnapsackApp.ViewModels"
xmlns:sp="clr-namespace:ScottPlot.Avalonia;assembly=ScottPlot.Avalonia"
x:Class="KnapsackApp.Views.MainWindow"
x:DataType="vm:MainWindowViewModel"
Title="Лабораторная работа №10 - Задача о рюкзаке"
Width="1000" Height="700">

    <Design.DataContext>
        <vm:MainWindowViewModel/>
    </Design.DataContext>

    <Grid ColumnDefinitions="300, *" Margin="10">
        <Grid Grid.Column="0" RowDefinitions="Auto, *">
            <Button Grid.Row="0"
                Content="Запустить бенчмарк"
                Command="{Binding StartBenchmarkCommand}"
                IsEnabled="{Binding !IsBusy}"
                HorizontalAlignment="Stretch"
                HorizontalContentAlignment="Center"
                Margin="0,0,10,10" Height="40"/>
            <TextBox Grid.Row="1"
                Text="{Binding Logs}"
                IsReadOnly="True" AcceptsReturn="True"
                TextWrapping="Wrap" Margin="0,0,10,0"
                FontFamily="Consolas, Courier New, monospace"/>
        </Grid>
        <Border Grid.Column="1" BorderBrush="Gray" BorderThickness="1">
            <sp:AvaPlot Name="AvaPlot1" />
        </Border>
    </Grid>
</Window>

```

Полный код: MainWindow.axaml.cs

```

using System.Collections.Generic;
using System.Linq;
using Avalonia.Controls;
using KnapsackApp.Services;
using KnapsackApp.ViewModels;
using ScottPlot;

namespace KnapsackApp.Views;

public partial class MainWindow : Window
{
    public MainWindow()
    {
        InitializeComponent();
        var vm = new MainWindowViewModel();
        DataContext = vm;
        vm.OnBenchmarksCompleted = DrawChart;
    }

    private void DrawChart(List<BenchmarkResult> results)
    {
        Avalonia.Threading.Dispatcher.UIThread.Post(() =>
        {
            AvaPlot1.Plot.Clear();

            double[] xs = results.Select(r =>
(double)r.Threads).ToArray();
            double[] yParallelFor = results.Select(r =>
(double)r.ParallelForTimeMs).ToArray();
            double[] yThreads = results.Select(r =>
(double)r.ThreadsTimeMs).ToArray();
            double baseTime = yParallelFor[0];
            double[] yIdeal = xs.Select(x => baseTime / x).ToArray();

            var s1 = AvaPlot1.Plot.Add.Scatter(xs, yParallelFor);
            s1.LegendText = "Parallel.For"; s1.LineWidth = 2; s1.MarkerSize = 7;

            var s2 = AvaPlot1.Plot.Add.Scatter(xs, yThreads);
            s2.LegendText = "Сырые потоки (Thread)"; s2.LineWidth = 2;
s2.MarkerSize = 7;

            var s3 = AvaPlot1.Plot.Add.Scatter(xs, yIdeal);
            s3.LegendText = "Идеальное ускорение"; s3.LineWidth = 2;
            s3.LinePattern = LinePattern.Dashed; s3.MarkerSize = 0;

            AvaPlot1.Plot.Title("Масштабируемость алгоритма (Задача о рюкзаке, 25
предметов)");
            AvaPlot1.Plot.XLabel("Количество потоков");
            AvaPlot1.Plot.YLabel("Время выполнения (мс)");
            AvaPlot1.Plot.Axes.AutoScale();
            AvaPlot1.Plot.ShowLegend();
            AvaPlot1.UserInputProcessor.Disable();
            AvaPlot1.Refresh();
        });
    }
}

```

3. СКРИНШОТЫ РАБОТЫ ПРИЛОЖЕНИЯ

Ниже представлены скриншоты, демонстрирующие работу приложения.

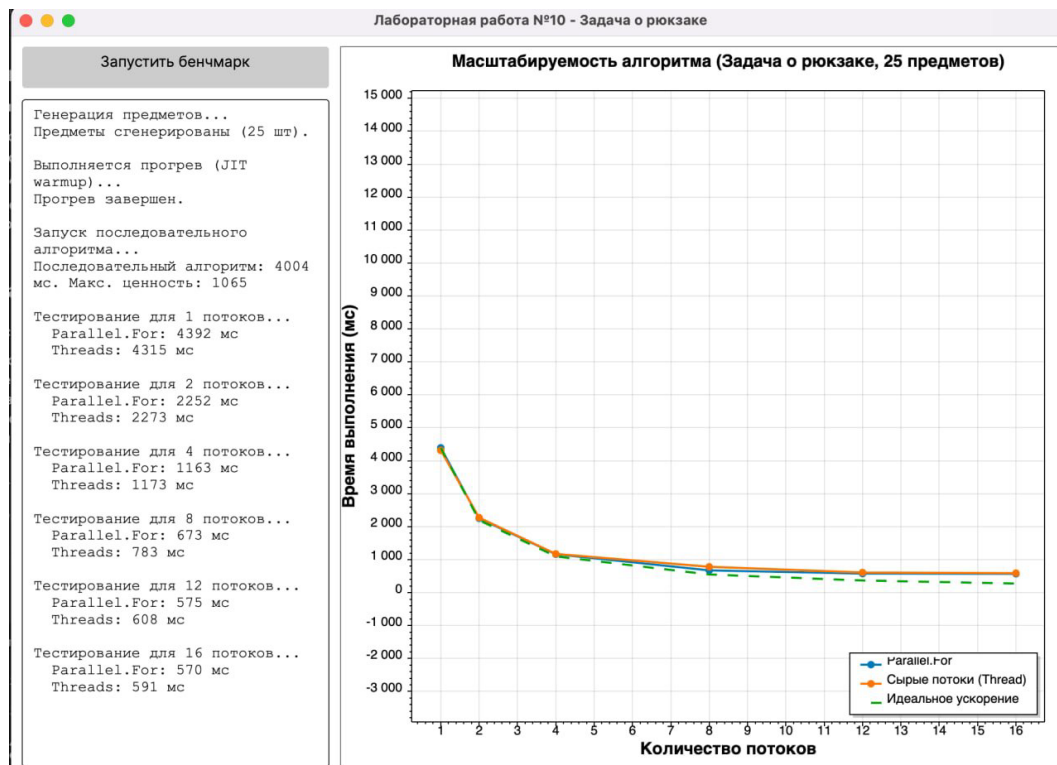


Рис. 1 — Главное окно и лог бенчмарка (полный диапазон оси Y)

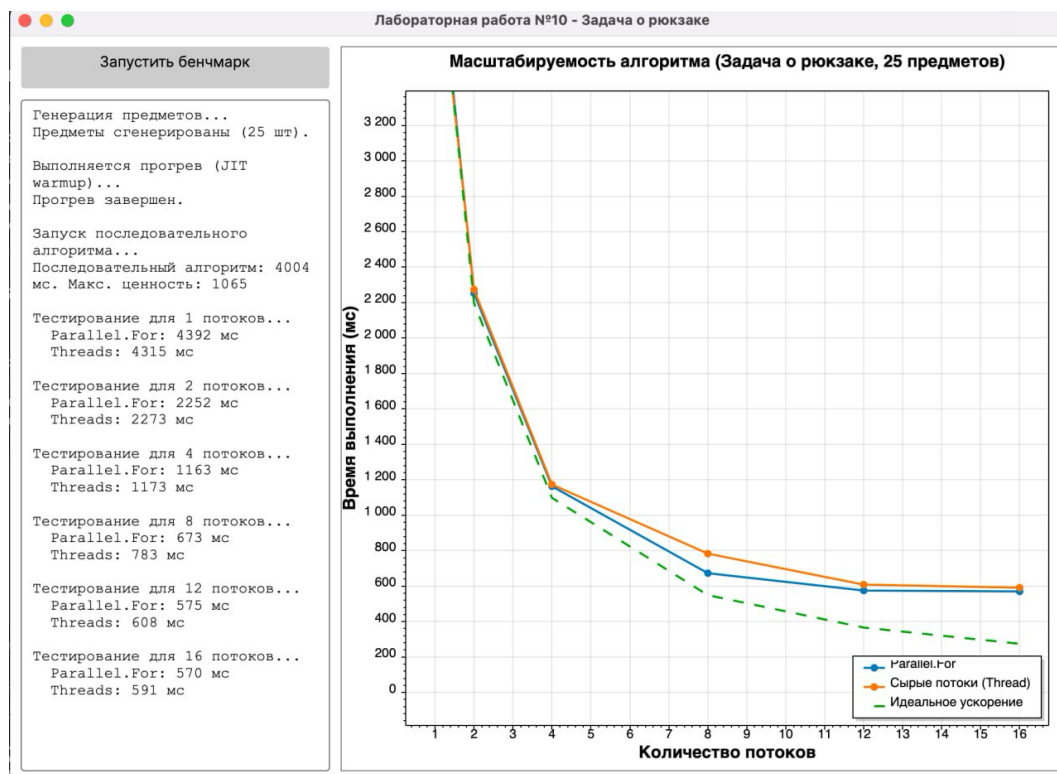


Рис. 2 — График масштабируемости (укрупнённый вид)

4. ВЫВОД

В ходе выполнения лабораторной работы было реализовано решение задачи о рюкзаке методом полного перебора среди $2^{25} = 33\,554\,432$ комбинаций. Задача решена тремя способами: последовательным перебором, параллельным через `Parallel.For` с `Thread-Local Storage` и через массив сырых потоков `Thread` с ручной декомпозицией данных и синхронизацией через `lock`.

Проведённые замеры времени выполнения для 1, 2, 4, 8, 12 и 16 потоков показали, что ускорение растёт практически линейно до числа потоков, соответствующего количеству логических ядер процессора. При дальнейшем увеличении числа потоков рост производительности замедляется из-за накладных расходов на переключение контекста. Построенный график наглядно демонстрирует отклонение реального ускорения от идеального, что согласуется с законом Амдала.

Сравнение `Parallel.For` и сырых потоков показало, что встроенный планировщик TPL работает эффективно благодаря использованию пула потоков и пакетной обработке итераций. Сырые потоки с ручной декомпозицией на крупные чанки демонстрируют сопоставимую производительность, однако требуют значительно большего объёма кода и ручного управления синхронизацией.